

Rate Limiting & Brute Force Protection Implementation

Date: May 10, 2026

Version: 1.0

Status: Implemented

Overview

This document describes the rate limiting and brute force protection mechanisms implemented to prevent account takeover, credential stuffing, and API abuse.

Protection Layers

1. **Login Rate Limiting** — Prevents brute force password attacks
 2. **Account Lockout** — Temporarily locks accounts after failed attempts
 3. **IP-Based Rate Limiting** — Blocks malicious IPs
 4. **API Rate Limiting** — Prevents API abuse and DoS
-

1. Login Rate Limiting & Account Lockout

Configuration

```
# application.properties
security.login.max-attempts=5
security.login.lockout-duration-minutes=15
```

Behavior

Per Email Address: - Maximum 5 failed login attempts within 15 minutes - After 5 failures, account is locked for 15 minutes - Successful login resets the counter

Per IP Address: - Maximum 15 failed login attempts (3x per-account limit) within 15 minutes - Prevents distributed attacks across multiple accounts - Successful login clears IP rate limit

Per IP (Request Rate): - Maximum 10 login requests per minute - Prevents rapid-fire brute force attempts - Uses token bucket algorithm (Bucket4j)

Implementation

Entity: LoginAttempt

```
@Entity
@Table(name = "login_attempts")
public class LoginAttempt {
    private UUID id;
    private String email;
```

```

    private String ipAddress;
    private boolean successful;
    private OffsetDateTime attemptedAt;
}

```

Service: LoginAttemptService - validateLoginAttempt(email, ip) — Checks if login is allowed - recordLoginAttempt(email, ip, success) — Records attempt - cleanupOldAttempts() — Scheduled cleanup (hourly)

Integration: AuthService.signIn()

```

public Map<String, Object> signIn(String email, String password, HttpServletRequest request) {
    String ipAddress = getClientIp(request);

    // 1. Validate rate limits BEFORE database lookup
    loginAttemptService.validateLoginAttempt(email, ipAddress);

    // 2. Authenticate user
    UserProfile user = userProfileRepository.findByEmailWithTenant(email)
        .orElseThrow(() -> {
            loginAttemptService.recordLoginAttempt(email, ipAddress, false);
            return AppException.notFound();
        });

    // 3. Validate password
    if (!passwordEncoder.matches(password, user.getPasswordHash())) {
        loginAttemptService.recordLoginAttempt(email, ipAddress, false);
        throw AppException.badRequest();
    }

    // 4. Record successful login
    loginAttemptService.recordLoginAttempt(email, ipAddress, true);
    return buildTokenResponse(user.getId(), tenantId);
}

```

Error Responses

All rate limit violations return:

```

{
  "status": 403,
  "code": "FORBIDDEN"
}

```

Note: Generic error prevents enumeration. Attacker cannot distinguish between: - Account locked - IP blocked - Invalid credentials

2. API Rate Limiting

Configuration

Authenticated Users: - 30 requests per minute per user - Applies to all authenticated endpoints

Unauthenticated IPs: - Public endpoints excluded (login, registration, health check) - Protected endpoints require authentication

Implementation

Filter: ApiRateLimitFilter

```
@Component
public class ApiRateLimitFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, ...) {
        String key = tenantContext.hasUser()
            ? "api:user:" + tenantContext.getCurrentUser()
            : "api:ip:" + getClientIp(request);

        var bucket = rateLimitService.resolveBucket(key, 100, Duration.ofMinutes(1));
        if (!bucket.tryConsume(1)) {
            // Return 429 Too Many Requests
            GlobalExceptionHandler.writeError(response,
                HttpStatus.TOO_MANY_REQUESTS,
                ErrorCode.RATE_LIMIT_EXCEEDED);
            return;
        }

        filterChain.doFilter(request, response);
    }
}
```

Service: RateLimitService - Uses Bucket4j token bucket algorithm - In-memory cache (ConcurrentHashMap) - Configurable capacity and refill rate

Error Response

```
{
  "status": 429,
  "code": "RATE_LIMIT_EXCEEDED"
}
```

Public Endpoints (Excluded from API Rate Limiting)

- /auth/** — Login, registration, password reset
- /health — Health check
- /tenants/register — Tenant registration
- /tenants/branding — Public branding endpoint
- /swagger-ui/** — API documentation

- /v3/api-docs/** — OpenAPI spec
- /github/webhook — GitHub webhook

Note: Login endpoints have their own specialized rate limiting (see section 1).

3. IP Address Extraction

X-Forwarded-For Support

When behind a reverse proxy (Cloudflare, nginx, etc.), the real client IP is in the X-Forwarded-For header:

```
private String getClientIp(HttpServletRequest request) {
    String xForwardedFor = request.getHeader("X-Forwarded-For");
    if (xForwardedFor != null && !xForwardedFor.isEmpty()) {
        return xForwardedFor.split(",")[0].trim(); // First IP in chain
    }
    return request.getRemoteAddr();
}
```

Security Note: Ensure your reverse proxy is configured to set X-Forwarded-For correctly and prevent header spoofing.

4. Database Schema

Migration: V005__login_attempts.sql

```
CREATE TABLE login_attempts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL,
    ip_address VARCHAR(45) NOT NULL,
    successful BOOLEAN NOT NULL DEFAULT FALSE,
    attempted_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_login_attempts_email_time ON login_attempts(email, attempted_at);
CREATE INDEX idx_login_attempts_ip_time ON login_attempts(ip_address, attempted_at);
CREATE INDEX idx_login_attempts_cleanup ON login_attempts(attempted_at);
```

Indexes

- idx_login_attempts_email_time — Fast lookup for per-email rate limiting
- idx_login_attempts_ip_time — Fast lookup for per-IP rate limiting
- idx_login_attempts_cleanup — Efficient cleanup of old records

Data Retention

- Login attempts older than 7 days are automatically deleted
 - Cleanup runs hourly via `@Scheduled` task
 - Prevents table bloat
-

5. Monitoring & Alerting

Log Events

Account Lockout:

`WARN: Account locked due to too many failed attempts: user@example.com`

IP Blocked:

`WARN: IP blocked due to too many failed attempts: 192.168.1.100`

Rate Limit Exceeded:

`WARN: API rate limit exceeded for: api:user:a1b2c3d4-...`

`WARN: Rate limit exceeded for IP: 192.168.1.100`

Recommended Alerts

1. High Failed Login Rate

- Threshold: >50 failed attempts per minute
- Action: Investigate potential credential stuffing attack

2. Multiple Account Lockouts

- Threshold: >10 accounts locked per hour
- Action: Check for distributed brute force attack

3. IP Block Frequency

- Threshold: >5 IPs blocked per hour
- Action: Review attack patterns, consider WAF rules

4. API Rate Limit Violations

- Threshold: >100 violations per hour
 - Action: Identify abusive users/bots
-

6. Testing

Manual Testing

Test Account Lockout:

```
# Attempt login 6 times with wrong password
for i in {1..6}; do
    curl -X POST http://localhost:8080/auth/login \
        -H "Content-Type: application/json" \
        -d '{"email":"test@example.com","password":"wrong"}'
done

# 6th attempt should return 403 FORBIDDEN
```

Test IP Rate Limiting:

```
# Rapid-fire login attempts
for i in {1..15}; do
    curl -X POST http://localhost:8080/auth/login \
        -H "Content-Type: application/json" \
        -d '{"email":"test@example.com","password":"test"}' &
done
wait

# Some requests should return 403 FORBIDDEN
```

Test API Rate Limiting:

```
# Authenticated user making 50 requests
TOKEN="<your-jwt-token>"
for i in {1..50}; do
    curl -H "Authorization: Bearer $TOKEN" \
        http://localhost:8080/tickets &
done
wait

# Requests after 30 should return 429 TOO_MANY_REQUESTS
```

Automated Testing

Unit Tests:

```
@Test
void shouldLockAccountAfterMaxAttempts() {
    // Record 5 failed attempts
    for (int i = 0; i < 5; i++) {
        loginAttemptService.recordLoginAttempt("test@example.com", "127.0.0.1", false);
    }

    // 6th attempt should throw exception
    assertThrows(AppException.class, () ->
```

```

        loginAttemptService.validateLoginAttempt("test@example.com", "127.0.0.1")
    );
}

```

Integration Tests:

```

@Test
void shouldReturnForbiddenWhenAccountLocked() {
    // Lock account
    for (int i = 0; i < 5; i++) {
        mockMvc.perform(post("/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"email\":\"test@example.com\",\"password\":\"wrong\"}"))
            .andExpect(status().isBadRequest());
    }

    // Next attempt should be forbidden
    mockMvc.perform(post("/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"email\":\"test@example.com\",\"password\":\"correct\"}"))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.code").value("FORBIDDEN"));
}

```

7. Configuration Tuning

Adjusting Limits

More Restrictive (High Security):

```

security.login.max-attempts=3
security.login.lockout-duration-minutes=30

```

Less Restrictive (User-Friendly):

```

security.login.max-attempts=10
security.login.lockout-duration-minutes=5

```

API Rate Limits

Edit `ApiRateLimitFilter.java`:

```

// Change from 30 requests/minute to 60 requests/minute
var bucket = rateLimitService.resolveBucket(key, 60, Duration.ofMinutes(1));

```

Login Request Rate

Edit LoginAttemptService.java:

```
// Change from 10 requests/minute to 5 requests/minute  
var bucket = rateLimitService.resolveBucket("login:" + ipAddress, 5, Duration.ofMinutes(1));
```

8. Cloudflare Integration

Edge Protection (Already Active)

Cloudflare provides: - DDoS protection - WAF (Web Application Firewall) - Bot detection - Geographic blocking - Challenge pages (CAPTCHA)

Application-Level Protection (This Implementation)

Our implementation adds: - Account-specific lockout (Cloudflare doesn't know about accounts) - Fine-grained per-user API rate limiting - Login attempt tracking and forensics - Custom business logic (e.g., different limits for different user roles)

Defense in Depth

Cloudflare (Edge)

- DDoS protection
- WAF rules
- Bot detection

Application (This Implementation)

- Account lockout
 - Per-user API rate limiting
 - Login attempt tracking
 - Custom business rules
-

9. Security Considerations

Timing Attacks

Mitigation: All rate limit violations return the same generic 403 FORBIDDEN error. Attacker cannot distinguish between: - Account locked - IP blocked - Invalid credentials - Email not found

Distributed Attacks

Mitigation: IP-based rate limiting (15 attempts per IP) prevents attackers from distributing attempts across multiple accounts from the same IP.

Credential Stuffing

Mitigation: Account lockout prevents automated credential stuffing even if attacker uses different IPs for each account.

Account Enumeration

Mitigation: Rate limiting applies BEFORE database lookup. Attacker cannot enumerate accounts by observing response times.

Lockout DoS

Risk: Attacker could intentionally lock legitimate user accounts.

Mitigation: - 15-minute lockout is short enough to minimize impact - Legitimate users can request password reset to unlock - Monitor for patterns of intentional lockouts

10. Troubleshooting

User Locked Out

Symptom: User reports “Access denied” when trying to log in.

Resolution: 1. Check login_attempts table:

```
sql SELECT * FROM login_attempts WHERE email = 'user@example.com' AND attempted_at > NOW() - INTERVAL '15 minutes' ORDER BY attempted_at DESC;
```

2. If locked, wait 15 minutes or manually clear:

```
DELETE FROM login_attempts WHERE email = 'user@example.com';
```

3. User can also request password reset (unlocks account).

Rate Limit Too Aggressive

Symptom: Legitimate users hitting rate limits.

Resolution: 1. Check logs for RATE_LIMIT_EXCEEDED events 2. Identify user/IP patterns 3. Adjust limits in application.properties or filter code 4. Consider whitelisting specific IPs (e.g., office network)

Memory Usage

Symptom: High memory usage from rate limit cache.

Resolution: - Current implementation uses in-memory cache (ConcurrentHashMap) - For high-traffic systems, consider Redis-backed cache - Bucket4j supports Redis integration

11. Future Enhancements

Planned Improvements

1. CAPTCHA Integration

- Add CAPTCHA after 3 failed attempts
- Prevents automated attacks while allowing legitimate users

2. Adaptive Rate Limiting

- Increase limits for trusted users (verified email, 2FA enabled)
- Decrease limits for suspicious behavior

3. Geolocation-Based Rules

- Block logins from unexpected countries
- Alert users of login from new location

4. Device Fingerprinting

- Track login attempts by device
- Alert on new device login

5. Redis-Backed Rate Limiting

- Distributed rate limiting across multiple servers
 - Persistent rate limit state
-

12. Dependencies

Added Libraries

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
</dependency>
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-core</artifactId>
  <version>8.10.1</version>
</dependency>
```

Why Bucket4j?

- Industry-standard token bucket algorithm
 - Thread-safe
 - Low overhead
 - Flexible configuration
 - Well-maintained
-

Summary

Rate limiting and brute force protection are now fully implemented with:

Account lockout after 5 failed attempts
IP-based rate limiting (15 attempts per IP)
Login request rate limiting (10 per minute per IP)
API rate limiting (30 requests per minute per user)
Automatic cleanup of old login attempts
Comprehensive logging and monitoring
Defense-in-depth with Cloudflare

Next Steps: - Monitor logs for attack patterns - Tune limits based on real-world usage - Consider CAPTCHA integration for additional protection

Document Version: 1.0

Last Updated: May 10, 2026

Author: Security Team